

Crossmark

PAPER

RECEIVED
dd Month yyyy
REVISED
dd Month yyyy

Random splits inflate the accuracy of composition-based superconductor T_c models: a leakage-aware evaluation replicated across two datasets

Gabriel Jaime Cardona Osorio 

Faculty of Engineering, Institución Universitaria Pascual Bravo, Medellín, Colombia

E-mail: gabriel.cardona@pascualbravo.edu.co**Keywords:** superconductivity, critical temperature, machine learning, materials informatics, data leakage, cross-validation, cluster sampling, design effect

Abstract

Composition-based machine-learning models routinely report $R^2 \approx 0.93$ and $\text{MAE} \approx 5$ K for superconducting critical temperature, but these figures are obtained by splitting datasets at the level of the individual compound. Superconductor datasets are not samples of independent compounds: they are assembled from doping series, so a random split places near-identical stoichiometric variants of the same chemistry on both sides of the partition. We quantify what that costs. Holding out whole chemical families—materials sharing a constituent-element set—inflates MAE by 52 % to 61 % and lowers R^2 by 0.071 to 0.081 across two independently curated datasets (21 263 and 12 440 compounds) and two featurizations, with the effect measured over five independent partitions. We then ask what the inflated number was measuring. Treating family identity as a clustering variable, the intraclass correlation of T_c within a family is 0.85 in both datasets, giving a cluster design effect of 3.6 to 5.5 and an effective sample size (3847 and 3470) within 14 % of the number of chemical families rather than the number of compounds. In-sample, family identity alone—no chemistry, no descriptors—accounts for 87.7 % and 88.4 % of the variance in T_c , which is the share of the target that a categorical label already explains. Screening utility, however, survives: under family-held-out validation the top-100 precision remains 87.0 % and 85.2 % against a 10 % base rate, while a family-mean lookup collapses to chance (11.6 % and 9.6 %). The correction is one extra cross-validation run, and we release deterministic family splits and code so it can be adopted as a reporting standard.

1 Introduction

Predicting the superconducting critical temperature T_c from chemical composition alone is one of the most-benchmarked regression tasks in materials informatics. The headline numbers are consistently strong: gradient-boosted trees and random forests on the Hamidieh featurization [1] or on Magpie descriptors [14] report $R^2 \gtrsim 0.92$ and MAE below 6 K, and deep architectures push the same metrics further [4, 5, 2]. Those numbers are quoted when arguing that machine learning is ready to guide the search for new superconductors, and models of this class have been used to propose candidates from text-mined phase diagrams [6], from Eliashberg-theory descriptors [7], and to screen hydride families [8, 9].

They are also obtained in a way that does not resemble the search. The datasets used are derived from SuperCon and are assembled from *doping series*: families of materials that share a constituent-element set and differ only in stoichiometry. A $\text{YBa}_2\text{Cu}_3\text{O}_{7-\delta}$ series contributes dozens of entries whose compositions differ in the third decimal place of the oxygen content and whose T_c values differ by a few kelvin. When such a dataset is split at random, near-identical twins of every test compound sit in the training set. The resulting metric measures interpolation within a series the model has already seen. A discovery campaign asks the opposite question: what is T_c for a chemistry that has never been synthesised?

The general form of this problem is well known. Meredig *et al* [3] introduced leave-one-cluster-out cross-validation for materials discovery and showed that extrapolative performance is systematically worse than the interpolative figure. Kapoor and Narayanan [10]

document leakage from non-independent samples as a recurring failure mode across machine-learning-based science. The Matbench efforts [12, 11] make the related point that strong regression metrics need not translate into discovery utility. What has been missing for superconductors specifically is a quantitative answer to three questions: which grouping is the right one for this domain, how large the gap is, and whether the gap is a property of one dataset or of the task.

This paper answers all three, and then asks a fourth question that the leakage framing does not by itself address. Sections 3.2 and 3.3 establish the gap and its stability. Section 3.5 asks what the inflated number was actually measuring, and finds that family identity alone—a variable containing no chemical information beyond a label—accounts for almost all of it. Section 3.6 shows that the practical screening application nonetheless survives, so the correct conclusion is a recalibration rather than a dismissal.

Our contributions are:

1. A domain-appropriate grouping variable: the constituent-element set, which corresponds one-to-one with the doping series that generate the redundancy, and which can be verified by inspection rather than assumed from an unsupervised clustering.
2. A measurement of the resulting gap, replicated across two independently curated datasets with two different featurizations and evaluated over five independent partitions.
3. A reframing of the gap in cluster-sampling terms: the intraclass correlation, design effect and effective sample size of each dataset, which quantify the inflation in a model-independent and protocol-independent way.
4. A ladder of nulls—family-mean lookup, element-presence indicators, full descriptors—that separates memorisation from learned chemistry, and shows that screening skill is genuine while most of the variance the conventional R^2 rewards is already available from a family label.
5. Deterministic, released family splits and the code to reproduce every number, so that the corrected protocol costs one command.

2 Data and methods

2.1 Datasets

Dataset A is the UCI Superconductivity Data Set [1]: 21 263 compounds with 81 statistical features derived from elemental properties (mean, weighted mean, geometric mean, entropy, range and standard deviation of atomic mass, ionisation energy, atomic radius, density, electron affinity, fusion heat, thermal conductivity and valence), accompanied by a per-element composition table giving the chemical formula of each entry.

Dataset A is used as distributed. It contains 66 rows that are exact duplicates of another row in every feature and in T_c , and 6093 rows (28.7%) whose 81 features coincide exactly with those of another row while T_c differs, because identical compositions yield identical composition-derived features. We do not remove them: they are the object of study, and the family grouping of section 2.2 places every such pair in the same fold.

Dataset B is the SuperCon-derived list curated by Stanev *et al* [2], which has 16 414 entries. We drop the 3966 entries with $T_c \leq 0$ (non-superconducting compositions) and 8 whose formula `pymatgen` cannot parse, yielding 12 440 compounds, and featurize them with the 132 Magpie descriptors [14] as implemented in `matminer` [13].

The two datasets share a common ancestry in SuperCon but differ in curation, in size, and entirely in featurization. Agreement between them is therefore evidence about the task rather than about a particular data-processing choice.

2.2 Chemical families

We define a *chemical family* as the set of constituent elements of a compound, irrespective of stoichiometry: $\text{Ba}_{0.2}\text{La}_{1.8}\text{Cu}_1\text{O}_4$ and $\text{Ba}_{0.1}\text{La}_{1.9}\text{Cu}_1\text{O}_4$ both belong to the family {Ba, La, Cu, O}. This is the grouping that corresponds to the doping series responsible for the redundancy, and unlike an unsupervised clustering it is inspectable: a reader can verify family membership from the formula.

The resulting concentration is severe (figure 1(a)). Dataset A contains 3365 families for 21 263 compounds, with 79.5% of all materials in families of five or more members; the three largest families hold 720, 358 and 323 compounds. Dataset B contains 3063 families for 12 440 compounds,

with 66.7% in families of five or more and a largest family of 131. Both datasets also carry a long tail of singletons (1169 and 1322 respectively).

Family labels are assigned with `pandas.factorize` applied *after* all row filtering. This ordering matters: applying it before a $T_c > 0$ mask leaves gaps in the label sequence, and because `GroupKFold` breaks ties by label order when assigning groups to folds, gapped and contiguous labellings produce different partitions of the same families. We return to this point in section 4.4.

2.3 Cross-validation protocols

We compare two protocols, identical in every respect except how the partition is formed:

- **Random:** `KFold(5, shuffle=True)`. Compounds are assigned to folds independently, so members of the same family routinely appear on both sides.
- **Chemical-family:** `GroupKFold(5)` with the family label as the grouping variable. Every family appears in exactly one fold, so no test compound shares a chemistry with any training compound.

For the robustness analysis of section 3.3 both protocols are repeated over five seeds. We use `GroupKFold(n_splits=5, shuffle=True, random_state=s)`, available since scikit-learn 1.6, so that the grouped partition genuinely varies with the seed; without `shuffle` the grouped splitter is deterministic and repeated runs differ only in model initialisation.

2.4 Models

We use the two tree ensembles that dominate reported results on this task, with hyperparameters fixed per dataset from prior grid searches and held constant across protocols. Dataset A: XGBoost [15] with 250 estimators, maximum depth 8, learning rate 0.07, subsample 0.9, column subsample 0.7, `gamma` 0, `reg_alpha` 0.01; random forest with 150 trees, maximum depth 25 and `sqrt` feature subsampling. Dataset B: XGBoost with 300 estimators, maximum depth 10, learning rate 0.05, subsample and column subsample 0.8, `hist` tree method; random forest with 150 trees and maximum depth 30, without feature subsampling. Both forests use `min_samples_leaf` 3 and `min_samples_split` 5. The two configurations differ because each was tuned against its own featurization; they are held fixed across protocols, which is what the random-versus-grouped comparison requires. Both were selected by grid search under *random* K -fold cross-validation in earlier work, i.e. under the protocol this paper argues is leaky. Because that could in principle bias the measured gap—a model tuned to interpolate may lose more when forced to extrapolate—section 3.4 repeats the comparison with hyperparameters selected by nested grouped cross-validation and over a 16-point grid. Regression metrics (MAE, RMSE, R^2) are computed on pooled out-of-fold predictions.

2.5 Nulls and the family-identity ceiling

To separate memorisation from learned chemistry we evaluate a ladder of three predictors under both protocols, on identical folds:

1. **Family-mean null.** No features at all: predict the mean T_c of the compound’s family as computed on the training fold, falling back to the global training mean for unseen families. This is a pure lookup table.
2. **Element-presence null.** The same XGBoost, trained on binary indicators of element presence only (86 indicators for Dataset A, 83 for Dataset B): does the compound contain Cu, does it contain O, and so on. All statistical and Magpie descriptors, and all stoichiometric information, are discarded.
3. **Full model.** The published featurization.

Separately, we quantify how much of the variance in T_c is attributable to family identity alone, treating the dataset as a clustered sample. With N compounds in k families of sizes n_i , a one-way random-effects ANOVA gives the intraclass correlation

$$\text{ICC} = \frac{\text{MS}_B - \text{MS}_W}{\text{MS}_B + (m_0 - 1)\text{MS}_W}, \quad m_0 = \frac{N - \sum_i n_i^2/N}{k - 1}, \quad (1)$$

Table 1. Pooled out-of-fold performance under random and chemical-family cross-validation. The family-mean null uses no features: it predicts the training-fold mean T_c of the compound’s family. Under the grouped protocol every test family is unseen by construction, so the null degenerates to a constant predictor and its R^2 is zero by definition—the informative comparison is the random column, where the same lookup reaches $R^2 = 0.80$ and 0.76 .

Model	Split	MAE (K)	RMSE (K)	R^2
<i>Dataset A</i> (Hamidieh/UCI, $N = 21\,263$, $k = 3365$ families)				
Family-mean null	Random	9.40	15.30	0.801
Family-mean null	Chemical-family	29.41	34.37	−0.007
XGBoost	Random	5.23	9.17	0.928
XGBoost	Chemical-family	8.31	13.06	0.855
Random forest	Random	5.40	9.47	0.923
Random forest	Chemical-family	8.68	13.47	0.845
<i>Dataset B</i> (Stanev/SuperCon, $N = 12\,440$, $k = 3063$ families)				
Family-mean null	Random	8.19	13.94	0.764
Family-mean null	Chemical-family	22.67	28.74	−0.003
XGBoost	Random	3.91	7.68	0.928
XGBoost	Chemical-family	6.17	10.99	0.853
Random forest	Random	4.33	8.31	0.916
Random forest	Chemical-family	6.52	11.56	0.838

where m_0 is Searle’s cluster size for unequal clusters. The corresponding design effect and effective sample size are

$$\text{deff} = 1 + (m - 1) \text{ICC}, \quad N_{\text{eff}} = N / \text{deff}, \quad (2)$$

which we report for both $m = \bar{m} = N/k$ and $m = m_0$. We also report the oracle bound $\eta^2 = 1 - \text{SS}_{\text{within}} / \text{SS}_{\text{total}}$ obtained from family means computed on the full dataset. We emphasise that η^2 is *not* a cross-validated R^2 : it is computed with knowledge of every family’s mean, including families that a cross-validated model would never see. It measures how much of the target a categorical label explains in-sample, and it is not commensurable with a cross-validated score in either direction: the grouped-CV figures of table 1 fall below it while the random-split figures exceed it. We therefore never take differences or ratios between η^2 and any R^2 , and plot it with distinct hatching throughout.

2.6 Screening metric

Discovery campaigns rank candidates rather than predicting each T_c exactly, so we also report top-100 precision: the fraction of the 100 highest-predicted test compounds whose true T_c exceeds a high- T_c threshold. We define that threshold as the 90th percentile of T_c *within the training fold*, so the base rate is 10 % by construction and no information from the test fold enters the evaluation criterion. Using the whole-dataset percentile instead changes the Dataset B grouped figure by 0.4 points, but it would let test compounds influence the threshold that judges them—the same class of leakage this paper is about. Precision is reported as the mean over folds with the between-fold standard deviation; features are not standardised.

2.7 Nearest-neighbour diagnostic

To confirm that the two protocols differ in the way the family construction predicts, we measure the Euclidean distance from each test compound to its nearest training compound in standardised feature space, under a random 80/20 hold-out and under a grouped hold-out of the same size.

3 Results

3.1 Reproducing the reported performance

Under the conventional protocol our models match the published range. A single random 80/20 split gives MAE = 5.14 K and $R^2 = 0.932$ on Dataset A, and MAE = 3.76 K with $R^2 = 0.934$ on Dataset B. These are the figures a reader of the literature would expect, and they establish that what follows is not an artefact of a weak baseline.

3.2 Holding out chemical families inflates the error

Table 1 reports both protocols for both models and both datasets, together with the family-mean null.

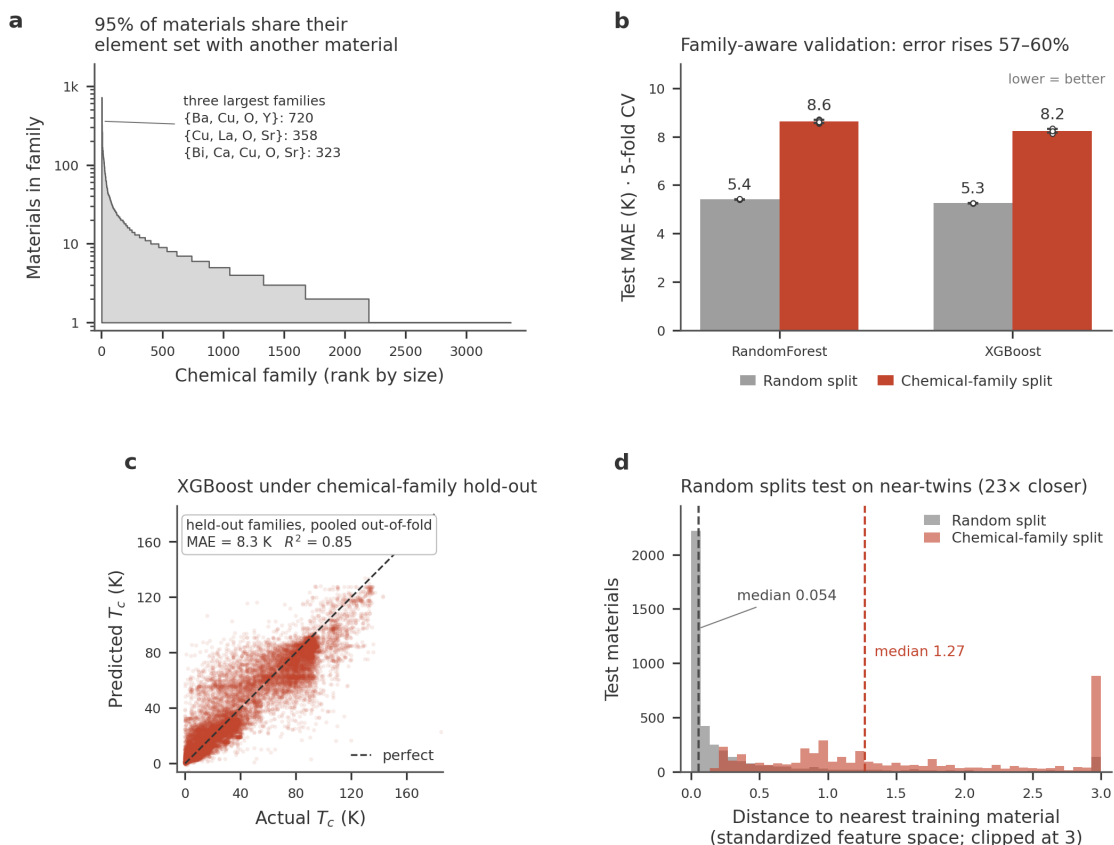
Anatomy of chemical data leakage in composition-based T_c prediction (Dataset A, $N = 21,263$)

Figure 1. Chemical leakage in Dataset A is structural rather than incidental: most materials share their exact element set with another material, so validation that ignores families tests models on near-duplicates. (a) Distribution of chemical-family size, a family being the set of materials with an identical constituent-element set ($n = 21\,263$ materials in $k = 3365$ families; the largest, {Ba, Cu, O, Y}, holds 720 doping variants). (b) Test MAE under both protocols for both models; bars are means over five independent family partitions, error bars the standard deviation across them, open circles the individual partitions. (c) Parity plot for XGBoost under chemical-family hold-out, pooling out-of-fold predictions of the deterministic partition ($n = 21\,263$ predictions, each from a model that never saw the test material’s family; MAE = 8.3 K, $R^2 = 0.85$, both cross-validated). (d) Distance from each test material to its nearest training material in standardised feature space for a single 80/20 hold-out per scheme ($n = 4253$ random and 4344 family-split test materials; distances clipped at 3 for display). The median test material is 23 times farther from its nearest training example under the family split, which is the mechanism behind the error increase in (b).

The pattern is the same in both datasets. Enforcing family-disjoint folds raises XGBoost’s MAE from 5.23 K to 8.31 K on Dataset A and from 3.91 K to 6.17 K on Dataset B, and lowers R^2 by 0.073 and 0.075. The random forest behaves identically. That two datasets with different curation, different size and completely different featurizations agree to within a few hundredths of R^2 indicates that the gap is a property of the prediction task, not of a particular data pipeline.

The nearest-neighbour diagnostic confirms the mechanism (figure 1(d)). Under a random hold-out the median distance from a test compound to its closest training compound is 0.054 in Dataset A; under a grouped hold-out of the same size it is 1.273, a factor of 23. For Dataset B the medians are 0.140 and 1.584, a factor of 11. The distance ratio understates the mechanism. Counting directly, 37.0(11)% of Dataset A test rows under five-fold random splitting have a training row with an *identical* 81-feature vector; under family-aware splitting the fraction is 0.0%. The random protocol is not evaluating extrapolation, nor even near-duplicate recall: for more than a third of its test set it is evaluating exact-duplicate recall, where the only thing left to predict is the measurement scatter of T_c within one composition. That scatter sets a floor: among the 8511 Dataset A rows that share their feature vector with another row, the mean absolute deviation of T_c from the within-group mean is 3.49 K, and XGBoost under random splitting reaches 4.83 K on those same rows. The floor is measured only on that 40% of the data and says nothing about the remainder; on the rows it does cover, most of the residual random-split error is label noise that no composition-based model can remove.

Table 2. Robustness over five independent partitions. Both arms use shuffled splitters with seeds 0 to 4, so the grouped partition itself varies between seeds and not only the model initialisation. Inflation is computed per seed and then averaged. Uncertainties are standard deviations across seeds.

		MAE random (K)		MAE grouped (K)		Inflation (%)		ΔR^2
A	XGBoost	5.25 $\pm$	0.01	8.24 $\pm$	0.07	56.9 $\pm$	1.6	-0.071 ± 0.003
A	Random forest	5.41 $\pm$	0.01	8.62 $\pm$	0.07	59.5 $\pm$	1.4	-0.076 ± 0.002
B	XGBoost	3.87 $\pm$	0.02	6.23 $\pm$	0.04	60.8 $\pm$	0.5	-0.081 ± 0.002
B	Random forest	4.32 $\pm$	0.01	6.58 $\pm$	0.02	52.5 $\pm$	0.6	-0.081 ± 0.002

Null-model decomposition: a featureless family-mean predictor separates memorization from genuine skill

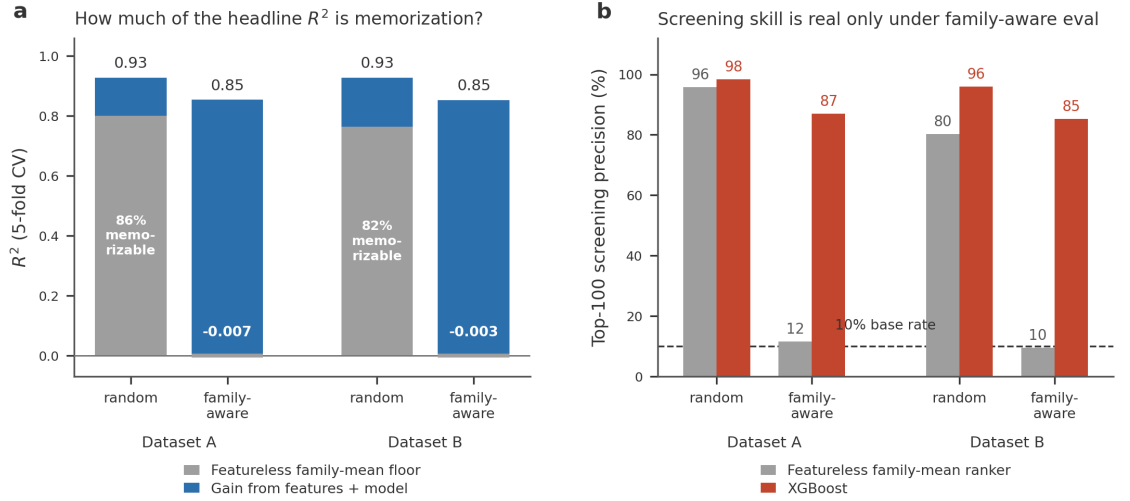


Figure 2. A featureless predictor that returns only each family’s mean T_c reproduces most of the headline performance under random splitting, locating that performance in family memorisation rather than in composition–property learning. (a) Decomposition of the cross-validated R^2 into the part the family-mean null already achieves (grey) and the increment contributed by features and model (blue), for both datasets under both protocols (Dataset A: $n = 21\,263$, $k = 3365$; Dataset B: $n = 12\,440$, $k = 3063$). Under random splitting the null alone reaches 86 % (A) and 82 % (B) of the full model’s R^2 ; under family-aware splitting it collapses to $R^2 \approx 0$ (-0.007 and -0.003 , drawn as a baseline stub) because a held-out family has no training mean to look up. All values are cross-validated; none is an oracle. (b) Top-100 screening precision of the same featureless ranker against XGBoost. Under random splitting the null ranks nearly as well as the model (95.8 % versus 98.4 % on A); under family-aware splitting it falls to 11.6 % (A) and 9.6 % (B), at or near the 10 % base rate, while XGBoost retains 87.0 % and 85.2 %. The threshold is the 90th percentile of training-fold T_c .

The gap itself is not a sampling artefact. Bootstrapping the pooled out-of-fold predictions (2000 resamples of the test rows) gives 95 % intervals for the MAE inflation of 55.1–63.0 % on Dataset A and 51.4–63.9 % on Dataset B, for XGBoost at seed 42; the random and grouped MAE intervals (5.13–5.33 K versus 8.18–8.44 K on A, 3.79–4.03 K versus 6.01–6.33 K on B) do not approach each other.

3.3 The gap is stable across partitions

A single partition could be unrepresentative, so we repeat both protocols over five independent partitions (table 2).

MAE inflation ranges from 52.5 % to 60.8 % across the four dataset–model combinations, and ΔR^2 from -0.071 to -0.081 (figure 3(a),(b)). The across-seed standard deviations are small—at most 1.6 percentage points of inflation—so the effect is not an artefact of a fortunate partition. We note that these dispersions do *not* grow appreciably when the partition is allowed to vary rather than only the model seed: pooled out-of-fold metrics average over the whole dataset, so redrawing the partition moves them very little. The honest statement of run-to-run uncertainty for a single reported figure is therefore the between-fold spread, which is considerably larger, and we quote it for the per-fold screening metric in section 3.6.

Table 3. Sensitivity of the random-versus-grouped gap to hyperparameter selection, XGBoost, pooled out-of-fold MAE (K). *Nested*: hyperparameters chosen by inner three-fold **GroupKFold** within each outer **GroupKFold(5)** fold. *Grid*: the same 16 configurations each scored under **KFold(5)** (shuffle, seed 42) and **GroupKFold(5)**; ranges are over the 16 points. Inflation is grouped MAE over random MAE minus one, computed with the same configuration in both arms.

	Dataset A			Dataset B		
	random	grouped	inflation (%)	random	grouped	inflation (%)
Published configuration	5.23	8.31	58.9	3.91	6.17	57.6
Nested grouped selection	5.11	8.37	63.9	4.07	6.23	53.1
Grid, depth 4 (range)	6.48–7.13	8.92–9.19	28.5–39.6	5.15–5.74	6.82–7.04	22.6–32.5
Grid, depth 6 (range)	5.50–5.93	8.46–8.53	43.4–55.2	4.33–4.68	6.34–6.47	38.3–46.4
Grid, depth 8 (range)	5.15–5.37	8.32–8.40	55.0–63.1	4.03–4.27	6.18–6.34	46.5–54.0
Grid, depth 10 (range)	5.05–5.11	8.31–8.52	63.4–68.7	3.87–4.07	6.17–6.31	52.4–60.7

3.4 The gap depends on model complexity, not on how the model was tuned

The hyperparameters of section 2.4 were chosen under random cross-validation, so a sceptic could ask whether the gap is partly an artefact of tuning under the leaky protocol. We test this two ways (table 3). First, nested grouped cross-validation: within each of the five outer **GroupKFold** folds, a 16-point XGBoost grid (maximum depth {4, 6, 8, 10}, learning rate {0.05, 0.1}, minimum child weight {1, 10}; 300 estimators) is selected by an inner three-fold **GroupKFold** on the training families only, then refit and scored on the outer test families. Second, the full grid evaluated under both protocols, which shows how the gap moves with complexity.

Tuning without leakage does not close the gap. The nested procedure selects the same complexity region as the original tuning (depth 8 to 10 in every outer fold of both datasets) and returns a grouped MAE of 8.37 K (A) and 6.23 K (B), 0.06 K *worse* than the published configuration scored under the same grouped protocol. The clean comparison—the same nested-selected configuration in both arms, so that only the partition protocol changes—gives an inflation of 63.9 % on A and 53.1 % on B.

What the grid reveals is where the gap comes from. Across all 16 configurations the grouped MAE moves within 0.9 K on A and 0.9 K on B, and within 0.2 K once depth is at least 6; the random-split MAE, by contrast, falls monotonically with depth: on A from 6.5–7.1 K at depth 4 to 5.05–5.11 K at depth 10, on B from 5.1–5.7 K to 3.87–4.07 K. The inflation therefore rises with depth: on A from 28.5–39.6 % at depth 4 to 63.4–68.7 % at depth 10, on B from 22.6–32.5 % to 52.4–60.7 %. Additional capacity buys fit to the within-family replicas that random splitting leaves in the training set; it buys almost nothing on unseen families. The single figure of ~58 % quoted elsewhere in this paper is thus the inflation for a competitively tuned model; for any configuration in the competitive region (depth ≥ 8) it lies between 47 % and 69 %, and it is not a consequence of having tuned under the leaky protocol.

A related robustness question is whether the gap depends on where the largest family lands. The {Ba, Cu, O, Y} family holds 720 Dataset A compounds, 64 % of them above 77 K, and as an indivisible block it makes whichever fold receives it the hardest. Replacing **GroupKFold** with **StratifiedGroupKFold** on five T_c quantile bins (three seeds) leaves the inflation at 57.2–61.9 % on A and 57.7–63.3 % on B, against 54.4–57.2 % and 60.3–61.3 % for the unstratified grouped splitter with the same seeds. The pooled gap does not depend on the placement of the largest family; what changes is the error of the single fold that receives it.

3.5 What the inflated number was measuring

The gap tells us the conventional figure is optimistic. It does not by itself say what the figure was measuring. Treating family identity as a clustering variable answers that (table 4).

Two facts stand out. First, the intraclass correlation is 0.85 in both datasets: knowing which doping series a compound belongs to determines most of its T_c . Second, the resulting effective sample size is essentially the number of chemical families. Dataset A’s 21 263 compounds carry the statistical information of 3847 independent observations against 3365 families, a ratio of 1.14; Dataset B’s 12 440 carry 3470 against 3063 families, a ratio of 1.13. This is the expected limit: as $\text{ICC} \rightarrow 1$, $\text{deff} \rightarrow m$ and hence $N_{\text{eff}} \rightarrow N/m = k$. At $\text{ICC} = 0.85$ both datasets sit within 14 % of it. The choice of cluster size makes almost no difference—using $\bar{m}$ instead of m_0 changes N_{eff} by less than 0.3 % in both datasets—so the conclusion does not rest on the equal-cluster approximation.

The consequence for the headline metric is shown in figure 4. In-sample, family identity alone

Table 4. Cluster structure of the two datasets. ICC is the intraclass correlation of T_c within a chemical family from a one-way random-effects ANOVA, equation (1); deff and N_{eff} follow equation (2), reported for both the mean cluster size $\bar{m}$ and Searle’s m_0 for unequal clusters. η^2 is the in-sample fraction of T_c variance attributable to family identity, computed from family means over the full dataset; it is not a cross-validated score and is not commensurable with the R^2 values in table 1.

Dataset	N	k	ICC	η^2	deff		N_{eff}		N_{eff}/k
					$\bar{m}$	m_0	$\bar{m}$	m_0	
A	21 263	3365	0.854	0.877	5.54	5.53	3836	3847	1.14
B	12 440	3063	0.846	0.884	3.59	3.59	3466	3470	1.13

Table 5. Top-100 precision under both protocols (per cent). The threshold is the 90th percentile of training-fold T_c , so the base rate is 10 %. Values are means over five folds with the between-fold standard deviation; the element-presence and full-model rows are additionally verified over five independent partitions in the text.

Predictor	Split	Dataset A		Dataset B	
Family-mean null	Random	95.8 $\pm$ 3.1		80.2 $\pm$ 1.5	
Family-mean null	Chemical-family	11.6 $\pm$ 10.9		9.6 $\pm$ 4.5	
Element presence	Random	95.6 $\pm$ 2.8		80.0 $\pm$ 3.1	
Element presence	Chemical-family	84.2 $\pm$ 15.1		66.8 $\pm$ 13.3	
Full model	Random	98.4 $\pm$ 1.2		96.0 $\pm$ 0.6	
Full model	Chemical-family	87.0 $\pm$ 11.0		85.2 $\pm$ 5.9	

accounts for $\eta^2 = 0.877$ of the variance in T_c on Dataset A and 0.884 on Dataset B. This is not a score any model achieves or competes with—it is computed with knowledge of every family’s mean, and the cross-validated figures in table 1 are not commensurable with it. What it measures is the share of the target that a categorical label already explains: on this task, seven-eighths of it.

The element-presence null makes a commensurable version of the same point, because it is cross-validated on identical folds. With no statistical or Magpie descriptors whatsoever, binary indicators of which elements are present reach $R^2 = 0.847$ and 0.846 under a random split, against 0.928 for the full featurization—about 91 % of the headline figure from element identity alone. The descriptor advantage does not widen when near-duplicates are removed; in R^2 it narrows, from 0.081 to 0.052 on Dataset A (0.803 versus 0.855 grouped) and from 0.082 to 0.074 on Dataset B (0.779 versus 0.853). Under both protocols and in both datasets, element identity alone recovers more than nine-tenths of the regression score the full featurization achieves. The screening metric behaves differently, and we return to it in section 3.6.

We stress the direction of this argument. It does not show that the models are learning nothing; section 3.6 shows the opposite for the task that matters. It shows that the conventional R^2 is a poor instrument for detecting what they learn, because most of the variance it rewards is available from a categorical label.

3.6 Screening utility survives, and is genuinely descriptor-driven

If most of the variance the regression metric rewards is already available from a family label, does the practical application survive? Table 5 reports top-100 precision for the three-rung null ladder.

Under family-held-out validation the full model retains 87.0 % precision on Dataset A and 85.2 % on Dataset B against a 10 % base rate. Screening utility is not destroyed by the corrected protocol: a campaign that synthesises the top-ranked 100 candidates from an unseen chemistry would still find high- T_c materials in more than four out of five attempts.

The null ladder shows that this skill is real rather than a lookup artefact (figure 2). The family-mean null, which dominates under a random split (95.8 % and 80.2 %), collapses to 11.6 % and 9.6 % under grouped validation—at or below the base rate, since it cannot consult an unseen family. Element-presence indicators retain substantial ranking power (84.2 % and 66.8 %), so a large part of the skill is attributable to chemical identity rather than to the descriptors’ physics. But the descriptors do contribute: measured over five independent partitions, discarding them costs 7.7 ± 1.0 points on Dataset A ($2.2\times$ the across-seed dispersion) and 14.4 ± 2.7 points on Dataset B ($9.9\times$), with the full model ahead in five seeds out of five in both datasets.

The two datasets differ in how much the descriptors buy: the gap in Dataset B is roughly twice that in Dataset A, a difference of 6.6 points at $3.3\times$ the across-seed dispersion with disjoint ranges.

Leakage-aware evaluation of T_c prediction: the chemical-data-leakage gap replicates on two independent datasets

(a, b) mean over 5 independent family partitions · (c) points are the 5 folds of one partition

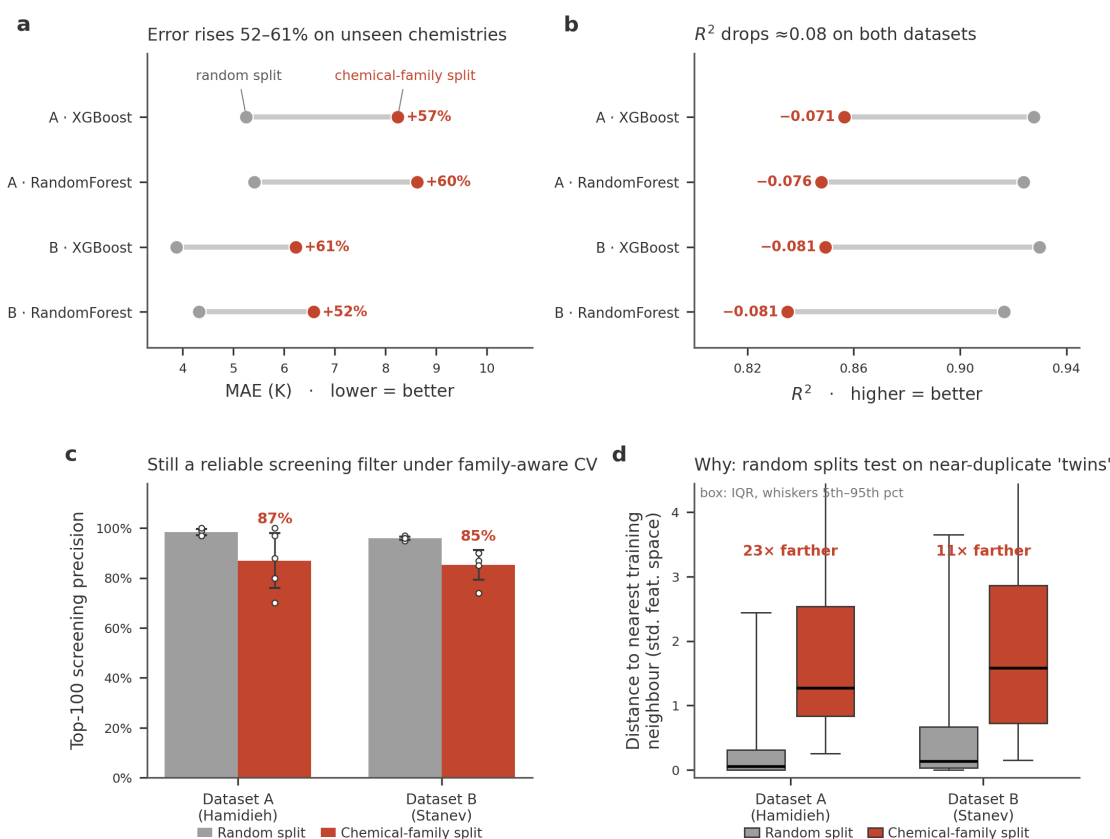


Figure 3. The gap is not an artefact of one dataset, one featurization or one model: it replicates across two independently constructed datasets, two model families and five independent partitions. (a) Test MAE and (b) R^2 for each dataset–model combination under both protocols. Points are means over five independent partitions; the annotation gives the paired per-seed MAE inflation and the signed change in R^2 . MAE rises 52 % to 61 % and R^2 falls by 0.071 to 0.081 in all four combinations. (c) Top-100 screening precision under both protocols; bars are the five-fold mean of the deterministic partition, error bars its between-fold standard deviation, open circles the individual folds. (d) Distance to the nearest training neighbour in standardised feature space for a single 80/20 hold-out per scheme (A: $n = 4253$ and 4344 ; B: $n = 2488$ and 2477 test materials); boxes show the interquartile range, whiskers the 5th–95th percentile. All quantities except the distances in (d) are cross-validated.

We do not have a confirmed mechanism for the asymmetry. Dataset A is the more family-concentrated of the two (79.5 % versus 66.7 % of compounds in families of five or more), which would plausibly make element identity a stronger cue there, but we have not tested that explanation and report the asymmetry as an observation.

4 Discussion

4.1 What the numbers mean

The single sentence that summarises this work is that *the row is not the sample*. Superconductor datasets are clustered samples of doping series, not simple random samples of compounds, and every consequence measured here follows from that. The 52 % to 61 % MAE inflation, the ICC of 0.85, the effective sample size that lands within 14 % of the family count, and the near-duplicate nearest-neighbour distances are four views of one fact.

The framing has a name outside materials informatics. Cluster sampling and its design effect are standard in survey statistics, where reporting a nominal N for a clustered sample is understood to be an error rather than a modelling choice. Stating the problem this way has two advantages over stating it as leakage: the diagnosis does not depend on which model is fitted or which cross-validation protocol is used, and the remedy—analyse at the level of the cluster—is already settled methodology.

Family identity alone explains $\sim 88\%$ of T_c variance in-sample; effective N falls to the number of families

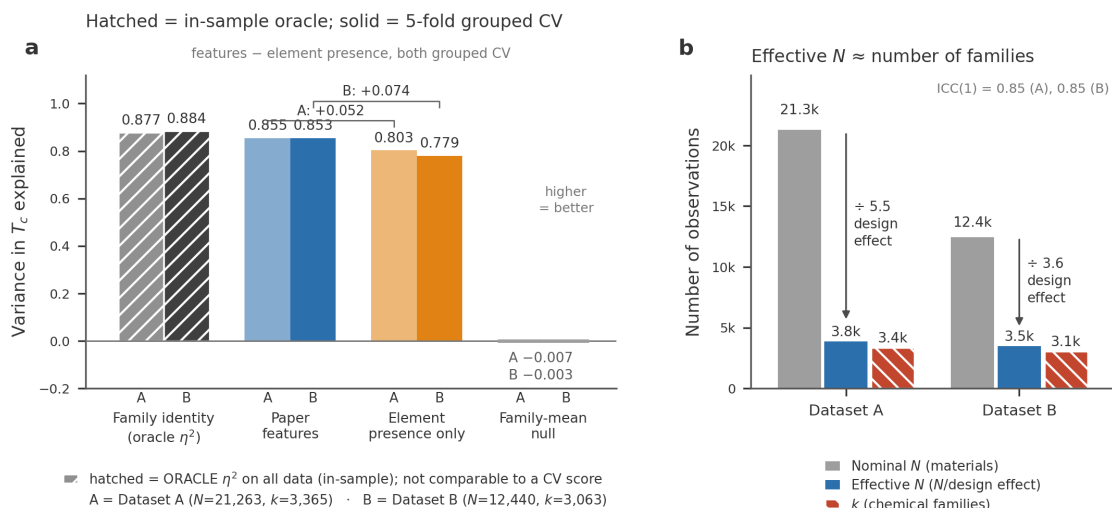


Figure 4. Family identity alone accounts in-sample for about seven-eighths of the variance in T_c , and the clustering that produces this also reduces the effective sample size to roughly the number of families. (a) Variance in T_c explained by four quantities, for Dataset A (light) and Dataset B (dark). The leftmost pair is an *oracle*: η^2 from a one-way random-effects ANOVA using true per-family means computed on *all* the data (0.877, 0.884). It is not a cross-validated score and not achievable by a model that has not seen the test family; it is drawn hatched for that reason and must not be read as a competitor to the bars beside it. The other three pairs are all cross-validated under grouped five-fold CV: the paper’s features (0.855, 0.853), a null trained only on binary element-presence indicators (0.803, 0.779) and the family-mean null (−0.007, −0.003). The brackets mark the only commensurable difference in the panel—features minus element presence, both grouped CV on identical folds—which is 0.052 on Dataset A and 0.074 on Dataset B. (b) The same clustering expressed as a design effect. Effective N is N divided by $1 + (m_0 - 1)\text{ICC}$ with Searle’s m_0 ; k is the family count (hatched, being a count of design units rather than observations). With $\text{ICC} = 0.85$ in both datasets the design effect is 5.5 (A) and 3.6 (B), so 21 263 materials carry about as much independent information as 3847, close to the 3365 families.

4.2 Relation to prior work

That extrapolative validation is harder than interpolative validation for materials discovery is established: Meredig *et al* [3] demonstrated it with leave-one-cluster-out cross-validation, and Kapoor and Narayanan [10] place it within a broader pattern of leakage-driven irreproducibility. The Matbench benchmarks [12, 11] similarly separate regression accuracy from discovery utility. Our contribution is not the observation that leakage exists but the specifics for this task: that the correct grouping variable for superconductors is the constituent-element set, which is chemically interpretable and directly verifiable rather than the output of an unsupervised clustering; that the resulting gap is 52 % to 61 % in MAE and replicates across two independently curated datasets and two featurizations; that the inflation admits a model-independent expression as a design effect; and that screening utility survives the correction.

4.3 Scope and limitations

Several boundaries should be stated plainly.

The family-mean null’s collapse to $R^2 \approx 0$ under grouped validation is a tautology, not an empirical finding: with every test family unseen, the lookup returns the global training mean, and a constant predictor has $R^2 = 0$ by definition. We report it for completeness because it calibrates the grouped column, but the informative half of that comparison is the random column, where the same lookup reaches 0.80 and 0.76 without using any feature.

The oracle η^2 is computed with full knowledge of family means, including families a cross-validated model never sees, so it is an in-sample descriptive quantity rather than a score any protocol could return. It is not a competitor to the cross-validated figures and not a bound on them—the random-split R^2 of 0.928 exceeds it, which is itself a symptom of the leakage this paper measures. We mark it distinctly in figure 4 and never combine it arithmetically with a cross-validated value.

The element-presence null removes stoichiometry and descriptor physics simultaneously, so it cannot attribute the resulting drop to either alone. A third rung—presence plus fractions, without Magpie descriptors—would separate them, and we have not run it.

We have measured only tree ensembles. Whether the gap widens with model capacity is a natural question for graph and deep composition models, and we expect it to, but we have not measured it and make no claim about it.

Finally, the corrected numbers are still retrospective. Family-held-out validation is a better proxy for discovery than a random split, but the decisive test is prospective: predicting compounds published after a cut-off date and measuring realised accuracy.

4.4 Reproducibility

The pipeline code, the featurized Dataset B checkpoint, the result tables underlying every table and figure in this paper, and the scripts that render the figures from those tables are released together for this revision (see Data availability). Dataset A is redistributed by UCI and is fetched by a script rather than bundled. Because a deterministic *procedure* turned out not to be a sufficient specification of a partition (see below), the partitions themselves are released as data: for every row of both datasets, its family identifier, its fold under the deterministic `GroupKFold(5)`, its fold under `GroupKFold` with shuffling for seeds 0 to 4, and its fold under `KFold` for the same seeds. The exact library versions are pinned, the SHA-256 of each data file is recorded, and one script checks every number in this text against the result file it came from.

Three implementation details materially affect the numbers and are documented here because they are not obvious from the protocol description. First, family labels must be assigned after row filtering: `GroupKFold` breaks ties by label order, and with thousands of singleton families a gapped labelling changes the fold of 8937 of the 12 440 Dataset B rows (1721 to 1897 per fold) relative to a contiguous labelling of the same families. The v1 release of our own Dataset B pipeline factorised family codes before removing the $T_c \leq 0$ rows and therefore evaluated a different partition from the one stored in its distributed checkpoint; every Dataset B number in this version was computed on the checkpoint partition, which is the one released row by row, and the pipeline now factorises after filtering. Second, the top-100 threshold must be stated explicitly, since the training-fold and whole-dataset definitions differ by 0.4 points on Dataset B. Third, top-100 precision is sensitive to the XGBoost version: the algorithmic changes across major versions alter the ranking of predictions even at fixed seed, and we therefore pin `xgboost==3.0.5` rather than a lower bound. The figures reported here were produced with Python 3.13, numpy 2.3.3, scikit-learn 1.7.2, xgboost 3.0.5 and matminer 0.9.2.

5 Conclusion

Composition-based T_c models are evaluated on datasets built from doping series, and splitting those datasets at the level of the compound measures interpolation within a series rather than generalisation to a new chemistry. Holding out whole chemical families inflates MAE by 52 % to 61 % and lowers R^2 by 0.071 to 0.081, consistently across two independently curated datasets and two featurizations.

Expressed as a clustered sample, the same fact reads more sharply: with an intraclass correlation of 0.85, these datasets carry the statistical information of roughly their number of chemical families—3847 and 3470 effective observations for 21 263 and 12 440 compounds—and family identity alone accounts in-sample for 87.7 % and 88.4 % of the variance in T_c . A cross-validated model trained only on which elements are present, with no descriptors at all, reaches $R^2 = 0.847$ and 0.846 under the conventional protocol, against 0.928 for the full featurization.

The practical conclusion is a recalibration rather than a dismissal. Screening precision under family-held-out validation remains 87.0 % and 85.2 % against a 10 % base rate, and the descriptors demonstrably contribute to it. What changes is the number that should be quoted: for Dataset A, MAE $\approx$ 8.3 K rather than 5.2 K. Obtaining it costs one additional cross-validation run, and we release the splits so that it costs one command.

Data availability

Code, the row-level cross-validation partitions of both datasets (`splits/`), the featurized Dataset B checkpoint, all result tables (`results_v2/`), the scripts that produced them and that render every figure (`code/v2/`), pinned dependency versions (`requirements.txt`) and SHA-256 checksums of the data files (`data/CHECKSUMS.sha256`) are available at <https://github.com/Freakscode/superconductor-tc-leakage-eval> under the MIT licence, at the tagged release accompanying this revision. `python code/v2/verify_manuscript.py` checks every number in this text against its result file. Dataset A is the UCI Superconductivity Data Set [1], retrieved by the included `get_datasetA.py`. Dataset B derives from the curated list of Stanev *et al* [2].

References

- [1] Hamidieh K 2018 A data-driven statistical model for predicting the critical temperature of a superconductor *Comput. Mater. Sci.* **154** 346–54
<https://doi.org/10.1016/j.commatsci.2018.07.052>
- [2] Stanev V, Oses C, Kusne A G, Rodriguez E, Paglione J, Curtarolo S and Takeuchi I 2018 Machine learning modeling of superconducting critical temperature *npj Comput. Mater.* **4** 29
<https://doi.org/10.1038/s41524-018-0085-8>
- [3] Meredig B, Antono E, Church C, Hutchinson M, Ling J, Paradiso S, Blaiszik B, Foster I, Gibbons B, Hattrick-Simpers J, Mehta A and Ward L 2018 Can machine learning identify the next high-temperature superconductor? Examining extrapolation performance for materials discovery *Mol. Syst. Des. Eng.* **3** 819–25 <https://doi.org/10.1039/c8me00012c>
- [4] Konno T, Kurokawa H, Nabeshima F, Sakishita Y, Ogawa R, Hosako I and Maeda A 2021 Deep learning model for finding new superconductors *Phys. Rev. B* **103** 014509
<https://doi.org/10.1103/PhysRevB.103.014509>
- [5] Zeng S, Zhao Y, Li G, Wang R, Wang X and Ni J 2019 Atom table convolutional neural networks for an accurate prediction of compounds properties *npj Comput. Mater.* **5** 84
<https://doi.org/10.1038/s41524-019-0223-y>
- [6] Court C J and Cole J M 2020 Magnetic and superconducting phase diagrams and transition temperatures predicted using text mining and machine learning *npj Comput. Mater.* **6** 18
<https://doi.org/10.1038/s41524-020-0287-8>
- [7] Xie S R, Quan Y, Hire A C, Deng B, DeStefano J M, Salinas I, Shah R S, Fanfarillo L, Lim J, Kim J, Stewart G R, Hamlin J J, Hirschfeld P J and Hennig R G 2022 Machine learning of superconducting critical temperature from Eliashberg theory *npj Comput. Mater.* **8** 14
<https://doi.org/10.1038/s41524-021-00666-7>
- [8] Sanna A, Cerqueira T F T, Fang Y-W, Errea I, Ludwig A and Marques M A L 2024 Prediction of ambient pressure conventional superconductivity above 80 K in hydride compounds *npj Comput. Mater.* **10** 44 <https://doi.org/10.1038/s41524-024-01214-9>
- [9] Wrona I A, Jarosik M W, Szcześniak R and Durajski A P 2024 A recipe for an effective selection of promising candidates for high-temperature superconductors among binary hydrides *Mater. Today Phys.* **46** 101499 <https://doi.org/10.1016/j.mtphys.2024.101499>
- [10] Kapoor S and Narayanan A 2023 Leakage and the reproducibility crisis in machine-learning-based science *Patterns* **4** 100804
<https://doi.org/10.1016/j.patter.2023.100804>
- [11] Riebesell J, Goodall R E A, Benner P, Chiang Y, Deng B, Lee A A, Jain A and Persson K A 2024 Matbench Discovery: A framework to evaluate machine learning crystal stability predictions *arXiv:2308.14920* <https://doi.org/10.48550/arXiv.2308.14920>
- [12] Dunn A, Wang Q, Ganose A, Dopp D and Jain A 2020 Benchmarking materials property prediction methods: the Matbench test set and Automatminer reference algorithm *npj Comput. Mater.* **6** 138 <https://doi.org/10.1038/s41524-020-00406-3>
- [13] Ward L, Dunn A, Faghaninia A, Zimmermann N E R, Bajaj S, Wang Q, Montoya J, Chen J, Bystrom K, Dylla M, Chard K, Asta M, Persson K A, Snyder G J, Foster I and Jain A 2018 Matminer: An open source toolkit for materials data mining *Comput. Mater. Sci.* **152** 60–9
<https://doi.org/10.1016/j.commatsci.2018.05.018>
- [14] Ward L, Agrawal A, Choudhary A and Wolverton C 2016 A general-purpose machine learning framework for predicting properties of inorganic materials *npj Comput. Mater.* **2** 16028
<https://doi.org/10.1038/npjcompumats.2016.28>
- [15] Chen T and Guestrin C 2016 XGBoost: A scalable tree boosting system *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining* 785–94
<https://doi.org/10.1145/2939672.2939785>